

AI ATTENTION VISUALIZER

OCR + Sentence Embeddings + Attention Mechanism

Interface	Streamlit Web Application
Input	Study-notes image (JPG, JPEG, PNG)
Main Technologies	Python, Tesseract OCR, Sentence Transformers, NumPy, Streamlit
Output	Extracted text, word attention bars, highest-attended word

1. Project Objective

The objective of this project is to build a simple AI application that can read text from a study-notes image, convert the extracted words into numerical embeddings, apply a basic scaled dot-product attention mechanism, and visually show which words receive relatively higher attention.

The project is designed to help students understand how different AI components can be connected in one application:

- OCR converts text in an image into machine-readable text.
- Sentence Transformers convert words into numerical embeddings.
- Attention calculates relationships between the word representations.
- Streamlit provides a simple interface to display the result.
- The visualization helps you to understand the idea of attention without displaying complicated matrices.

2. Problem Statement

Students often learn OCR, embeddings, and attention as separate concepts. This project combines them into one small application. A user uploads an image containing study notes. The application extracts the text, processes the words, calculates attention scores, and displays a bar for each word.

3. Learning Outcomes

- Understand the basic purpose of OCR.
- Understand what an embedding is and why text is converted into numbers.
- Understand the basic idea of Query, Key, Value and scaled dot-product attention.
- Understand how attention weights can be visualized.
- Build a simple Streamlit AI application.
- Connect multiple Python modules into one project.
- Run and test an AI application locally in VS Code.

4. Project Flow

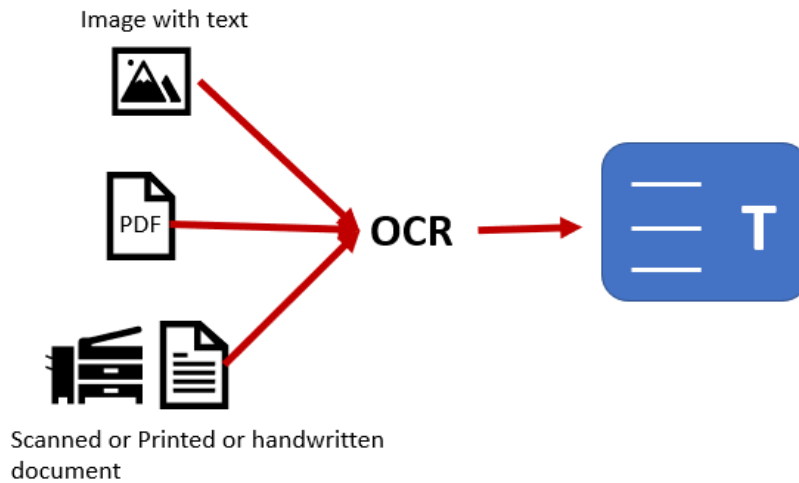
The complete application follows this flow:

Image → OCR → Extract Words → Embeddings → Attention → Word Attention Bars

What is OCR?

Optical Character Recognition.

Using optical character recognition (OCR) technology, a text image can be transformed into a machine-readable text format. When this technology is integrated into [Image to Text converter](#), it helps extract text from images of written text, including scanned documents, pictures, and other images.



5. System Architecture

Stage	Technology	Input	Output
1. Image Upload	Streamlit + Pillow	JPG/JPEG/PNG	Image object
2. OCR	Tesseract + pytesseract	Image	Extracted text
3. Word Processing	Python	Extracted text	Clean word list
4. Embeddings	Sentence Transformers	Words	384-dimensional vectors
5. Attention	NumPy	Embeddings	Attention weights
6. Visualization	Streamlit	Attention scores	Progress bars + top word

6. Technologies and Libraries

Technology / Library	Purpose	Installation
Python	Main programming language	Install Python 3.x
Streamlit	Build the web interface	pip install streamlit
Pillow	Open and display images	pip install pillow
pytesseract	Connect Python with Tesseract OCR	pip install pytesseract

Tesseract OCR	Extract text from images	Install separately on Windows
sentence-transformers	Generate text embeddings	pip install sentence-transformers
NumPy	Matrix and attention calculations	pip install numpy

7. Prerequisites

- Windows computer.
- Python 3.x installed.
- VS Code installed.
- Internet connection for installing Python packages and downloading the pretrained embedding model the first time.
- Tesseract OCR installed separately.
- Basic knowledge of Python is recommended.

8. Installation and Setup

8.1 Create the Project Folder

Create a folder named:

```
AI-Attention-Visualizer
```

Open this folder in VS Code.

8.2 Create the Project Files

```
AI-Attention-Visualizer/
|
├── app.py
├── ocr.py
├── embedding.py
├── attention.py
└── requirements.txt
```

8.3 Install Python Packages

Open the VS Code Terminal and run:

```
pip install streamlit numpy pillow pytesseract sentence-transformers
```

Alternatively, after creating requirements.txt, run:

```
pip install -r requirements.txt
```

8.4 Install Tesseract OCR on Windows

Tesseract is an OCR engine used to recognize text from images. The Python package pytesseract is only a Python wrapper; the Tesseract application itself must also be installed.

- Install Tesseract OCR for Windows.
- A common installation location is: C:\Program Files\Tesseract-OCR\
- Check that tesseract.exe exists in that folder.
- The path is configured in ocr.py.

If Tesseract is installed in another folder, change the path in ocr.py accordingly.

9. requirements.txt

```
streamlit
numpy
pillow
pytesseract
sentence-transformers
```

10. Module 1 – OCR (ocr.py)

This module receives an image and uses Tesseract OCR to extract the text. The Tesseract path below is the common Windows installation path.

```
import pytesseract

pytesseract.pytesseract.tesseract_cmd = (
    r"C:\Program Files\Tesseract-OCR\tesseract.exe"
)

def extract_text(image):
    return pytesseract.image_to_string(image)
```

Explanation:

- pytesseract connects Python to the Tesseract OCR engine.
- image_to_string() reads text from the image.
- The function returns the extracted text.

11. Module 2 – Embeddings (embedding.py)

The application uses the pretrained Sentence Transformer model all-MiniLM-L6-v2. It creates a 384-dimensional numerical representation for each input word.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer(
    "all-MiniLM-L6-v2"
)

def create_embeddings(words):
    embeddings = model.encode(words)
    return embeddings
```

What is an embedding?

An embedding is a numerical representation of text. Instead of processing a word only as characters, an AI model represents it as a vector of numbers. Similar meanings can be represented by related vectors.

Important note for students:

all-MiniLM-L6-v2 is mainly a sentence-embedding model. In this project, it is applied to individual words to keep the demonstration simple. This is useful for learning embeddings, but it is not the same as extracting the model's internal Transformer token-level attention.

12. Module 3 – Attention Mechanism (attention.py)

```
import numpy as np

def softmax(x):
    exp_x = np.exp(
        x - np.max(x, axis=-1, keepdims=True)
    )
    return exp_x / np.sum(
        exp_x,
        axis=-1,
        keepdims=True
    )

def calculate_attention(X):
    d_model = X.shape[1]
    d_k = 64

    np.random.seed(42)

    WQ = np.random.randn(d_model, d_k)
    WK = np.random.randn(d_model, d_k)
    WV = np.random.randn(d_model, d_k)

    Q = X @ WQ
    K = X @ WK
    V = X @ WV

    scores = Q @ K.T

    scaled_scores = scores / np.sqrt(d_k)

    attention_weights = softmax(scaled_scores)

    word_scores = attention_weights.mean(axis=0)

    return word_scores
```

Attention formula:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

Simple explanation:

- X represents the embedding vectors.
- Q (Query), K (Key), and V (Value) are created using matrix multiplication.
- $\mathbf{Q}\mathbf{K}^T$ calculates relationships between the representations.

- Dividing by $\sqrt{d_k}$ scales the scores.
- Softmax converts the scores into attention weights.
- The application averages attention received by each word to obtain one display score per word.
- The bars in the Streamlit application show these relative scores.

Important technical note: the Q, K and V projection matrices in this classroom version are randomly initialized. Therefore, the attention visualization demonstrates the mechanics of applying attention to embeddings; it should not be interpreted as a trained model's learned understanding of which study word is semantically most important.

13. Module 4 – Streamlit Application (app.py)

```
import streamlit as st
import numpy as np
from PIL import Image

from ocr import extract_text
from embedding import create_embeddings
from attention import calculate_attention

st.title("🧠 AI Attention Visualizer")

st.write(
    "Upload an image to extract text "
    "and visualize attention scores."
)

file = st.file_uploader(
    "Upload an image",
    type=["jpg", "jpeg", "png"]
)

if file:
    image = Image.open(file)
    st.image(image, width=500)

    text = extract_text(image)

    st.subheader("📄 Extracted Text")

    if not text.strip():
        st.error("No text found in the image.")
        st.stop()

    st.write(text)

    words = text.split()

    words = [
        word.strip(".,!?:;()[]{}")
        for word in words
    ]

    words = [
        word
        for word in words
        if len(word) > 2
    ]

    words = words[:20]
```

```

embeddings = create_embeddings(words)

scores = calculate_attention(embeddings)

st.subheader("🧠 Word Attention")

display_scores = scores / scores.max()

for word, score in zip(words, display_scores):
    st.write(f"***{word}***")
    st.progress(float(score))

top_index = np.argmax(scores)
top_word = words[top_index]

st.success(
    f"🌟 Highest Attention: **{top_word}**"
)

```

14. How the Complete Application Works

Step	What happens?	Result
Step 1	User uploads an image.	Streamlit receives the image.
Step 2	Image is opened using Pillow.	The image becomes a Python image object.
Step 3	OCR is performed.	Tesseract extracts the text.
Step 4	Text is split into words.	Words are cleaned and words shorter than 3 characters are removed.
Step 5	Maximum 20 words are selected.	This keeps the classroom demonstration simple and fast.
Step 6	Embeddings are generated.	all-MiniLM-L6-v2 creates numerical vectors.
Step 7	Attention is calculated.	Q, K and V are generated and scaled dot-product attention is applied.
Step 8	Scores are normalized for display.	Each word receives a relative progress-bar value.
Step 9	Highest score is identified.	The word with the highest calculated score is displayed.

15. Running the Application

- Open the AI-Attention-Visualizer folder in VS Code.
- Open Terminal → New Terminal.
- Make sure all packages are installed.

- Make sure Tesseract OCR is installed and its path in ocr.py is correct.
- Run the application using the command below.

```
streamlit run app.py
```

Streamlit will display a local web address in the terminal. Open that address in a browser.

16. Testing the Project

Use a clear study-notes image containing printed English text.

- Test 1: Upload a clear image with a short paragraph.
- Test 2: Upload an image with a heading and body text.
- Test 3: Upload an image containing handwritten text and observe OCR limitations.
- Test 4: Upload an image with no readable text and verify that the application shows 'No text found'.
- Test 5: Upload an image with more than 20 words and verify that only the first 20 cleaned words are visualized.

17. Expected Output

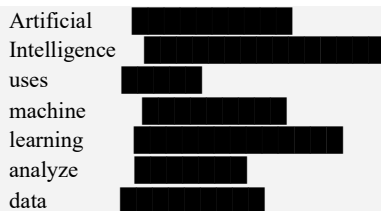
- Uploaded study-notes image is displayed.
- OCR-extracted text is displayed.
- A Word Attention section appears.
- Each selected word is displayed with a progress bar.
- The word with the highest calculated attention score is highlighted.

18. Example

Suppose the OCR extracts:

Artificial Intelligence uses machine learning to analyze data.

The application converts the selected words into embeddings and calculates attention scores. The final display may look conceptually like:



The exact scores depend on the embeddings and the attention calculation. The highest bar is simply the word receiving the highest calculated attention score in this demonstration.

19. Why Do We Need Each Component?

Component	Why it is used	Simple meaning
-----------	----------------	----------------

OCR	Reads text from an image	Image → Text
Embedding	Converts text into numerical vectors	Text → Numbers
Attention	Measures relationships between representations	Numbers → Attention weights
Streamlit	Displays the application	Python → Web UI

20. Common Errors and Solutions

Error / Problem	Solution
'python' is not recognized	Install Python and make sure Python is added to PATH. Restart VS Code after installation.
No module named streamlit	Run: pip install streamlit
No module named sentence_transformers	Run: pip install sentence-transformers
No module named pytesseract	Run: pip install pytesseract
TesseractNotFoundError	Install Tesseract OCR and verify the tesseract.exe path in ocr.py.
No text found in the image	Use a clearer image with larger, printed text and good lighting/contrast.
Model download is slow on first run	The pretrained embedding model is downloaded the first time. Allow the download to finish.
Progress bar issue	Use a numeric value between 0 and 1 in st.progress(). The current app normalizes the scores before display.

21. Project Limitations

- OCR quality depends on image clarity, font, orientation, and language.
- Only the first 20 cleaned words are used in this simple demonstration.
- The MiniLM model is being used at word level for educational simplicity.
- The Q, K and V projection matrices are randomly initialized and are not learned from a training dataset.
- Therefore, the highest-attention word should not be treated as a reliable semantic importance detector.
- The application demonstrates the attention mechanism rather than reproducing the internal attention maps of a pretrained Transformer.

22. Possible Future Enhancements

- Use a Transformer model that exposes actual token-level attention weights.
- Add sentence-level summarization.
- Highlight important words directly on the OCR text.
- Use OCR bounding boxes to highlight words on the original image.
- Support Tamil and other languages with suitable OCR models.

- Add keyword extraction and topic classification.
- Allow users to download the attention report.
- Add a study-notes summarizer.
- Add a question-answering feature for the uploaded notes.

23. Viva / Discussion Questions

1. What is OCR?
2. Why is OCR required in this project?
3. What is an embedding?
4. Why do we convert text into numerical vectors?
5. What is the purpose of Query, Key and Value?
6. What is scaled dot-product attention?
7. Why is softmax used?
8. What does an attention weight represent?
9. Why are the attention bars normalized before displaying them?
10. What is the role of Streamlit?
11. What is the difference between OCR and NLP?
12. Why is Tesseract installed separately from pytesseract?
13. What is all-MiniLM-L6-v2?
14. Does the highest-attention word always mean the most important word?
15. How could this project be improved to show true Transformer attention?

24. Student Implementation Activity

Students can complete the project in the following stages:

Stage	Task	Expected Result	Suggested Time
1	Create project folder and files	Correct folder structure	10 min
2	Install Python packages	All imports work	15 min
3	Configure Tesseract	OCR can read an image	15 min
4	Test OCR	Text appears from image	15 min
5	Test embeddings	Word embeddings are generated	15 min
6	Implement attention	Attention scores are produced	20 min
7	Build Streamlit UI	Application runs in browser	20 min
8	Test and document	Working project + screenshots	20 min

25. GitHub Repository

Suggested repository name:

```
AI-Attention-Visualizer
```

Suggested GitHub description:

```
A beginner-friendly AI application that extracts text from images using OCR, converts words into embeddings, and visualizes attention scores using a simple attention mechanism.
```

26. Suggested README Content

```
# AI Attention Visualizer
```

```
A beginner-friendly AI application that extracts text from images using OCR, converts words into embeddings, and visualizes attention scores using a simple attention mechanism.
```

```
## Technologies Used
```

- Python
- Streamlit
- Tesseract OCR
- Sentence Transformers
- NumPy
- Pillow

```
## Project Flow
```

```
Image → OCR → Word Embeddings → Attention → Visualization
```

```
## Installation
```

```
pip install -r requirements.txt
```

```
## Run
```

```
streamlit run app.py
```

```
## Features
```

- Upload study-note images
- Extract text using OCR
- Generate word embeddings
- Calculate attention scores
- Visualize word attention
- Identify the highest-attended word

```
## Note
```

```
Tesseract OCR must be installed separately on Windows.
```

27. Conclusion

The AI Attention Visualizer is a compact educational project that demonstrates how image processing, OCR, text embeddings, attention, and a web interface can be combined in one application. The project is intentionally simple so that students can focus on understanding the flow from image input to attention visualization rather than getting lost in large models or complex mathematical implementations.